



Universidad Politécnica de Chiapas

Universidad Politécnica de Chiapas
Ingeniería En Tecnologías De La Información E Innovación
Digital.

Fecha:

05 de febrero del 2026

Nombre:

Derek Alejandro Ortiz Tovilla

José Juan Ramos Cabrera

Alexander Jesús Jiménez León

Actividad:

Examen: Diseño de sistemas y Base de Datos

Matricula:

243742

243708

243713

Materia:

Bases de Datos Avanzada

Maestro:

Ramsés Camas Nájera

1. ¿Por qué escogieron este caso?

Lo elegimos porque representaba el **mayor desafío técnico y realista**. No queríamos un sistema básico, sino uno que nos exigiera resolver tres problemas complejos:

- **Integridad Transaccional:** Manejar inventarios afectados simultáneamente por compras y ventas.
- **Relaciones N:N Reales:** Resolver la logística de múltiples proveedores en un pedido y múltiples pedidos en una ruta.
- **SQL Avanzado:** Aplicar lógica matemática en Vistas para reportes críticos, superando el nivel básico de almacenamiento."

2. ¿Por qué lo diseñaron así?

El diseño prioriza la **Normalización y la Trazabilidad** en tres decisiones clave:

1. **Tabla ASIGNACION_PEDIDOS_RUTA:** Nos permite registrar **historial de intentos** e incidencias específicas (ej. 'Casa cerrada') sin ensuciar los datos del pedido original.
2. **Estados Calculados:** No guardamos el estado manualmente para evitar errores humanos. El sistema **deduce el estado** dinámicamente (ej. *Si hay firma = Entregado*), garantizando integridad absoluta.
3. **Separación Entradas/Salidas:** Usamos tablas distintas para Proveedores y Clientes porque sus datos de auditoría son incompatibles, manteniendo una estructura limpia y escalable.

2) Restricciones Identificadas (Adaptadas: Adiós a los CHECKs de Estado)

A. Restricciones de Integridad Crítica

Para que el "Estado Calculado" no falle, las fechas y firmas se vuelven obligatorias en ciertos contextos.

1. **Fechas de Transición:**
 - Fecha_Programada en RUTAS debe ser NOT NULL.

- Fecha_Solicitud en PEDIDOS debe ser NOT NULL.

2. Unicidad Temporal (Para no calcular doble estado):

- Un vehículo no puede tener dos mantenimientos abiertos el mismo día.
UNIQUE (ID_Vehiculo, Fecha) en MANTENIMIENTOS.

B. Restricciones de Unicidad (UNIQUE) - Se mantienen

3. **RFC Proveedores:** CONSTRAINT UQ_Prov_RFC UNIQUE (RFC).
4. **RFC Comercios:** CONSTRAINT UQ_Comercio_RFC UNIQUE (RFC).
5. **Placas:** CONSTRAINT UQ_Placas UNIQUE (Placas).
6. **Licencias:** CONSTRAINT UQ_Licencia UNIQUE (Licencia).

C. Restricciones de Negocio (CHECK) - Se mantienen

7. **Precios:** CHECK (Precio_Venta > Precio_Compra).
8. **Stock:** CHECK (Stock_Actual >= 0).
9. **Cantidades:** CHECK (Cantidad > 0) (En pedidos y entradas).
10. **Licencia Vigente:** CHECK (Vencimiento_Lic > CURRENT_DATE).

3) Queries SQL de Ejemplo (Dinámicos)

A. Pedidos confirmados que aún no tienen ruta asignada, con datos del comercio.

```
SELECT

    p.ID_Pedido,

    c.Nombre AS Comercio,

    z.Nombre AS Zona,

    p.Fecha_Solicitud,

    SUM(dp.Cantidad * prod.Peso_Unitario) AS Peso_Total_Pedido_KG

FROM PEDIDOS p

JOIN COMERCIOS c ON p.ID_Comercio = c.ID_Comercio
```

```

JOIN ZONAS z ON c.ID_Zona = z.ID_Zona

JOIN DETALLE_PEDIDO dp ON p.ID_Pedido = dp.ID_Pedido

JOIN PRODUCTOS prod ON dp.ID_Producto = prod.ID_Producto

WHERE p.Estado = 'Confirmado'

    AND NOT EXISTS (

        SELECT 1 FROM ASIGNACION_PEDIDOS_RUTA apr

        WHERE apr.ID_Pedido = p.ID_Pedido

    )

GROUP BY p.ID_Pedido, c.Nombre, z.Nombre, p.Fecha_Solicitud

ORDER BY p.Fecha_Solicitud ASC;

```

B. Peso total asignado a una ruta vs capacidad del vehículo.

```

SELECT

    r.ID_Ruta,

    r.Fecha_Programada,

    v.Placas,

    v.Capacidad_KG,

    COALESCE(SUM(dp.Cantidad * prod.Peso_Unitario), 0) AS
Peso_Cargado_Total,

    CASE

        WHEN SUM(dp.Cantidad * prod.Peso_Unitario) > v.Capacidad_KG
    THEN 'PELIGRO: SOBRECARGA'

        WHEN SUM(dp.Cantidad * prod.Peso_Unitario) > (v.Capacidad_KG *
0.9) THEN 'Optimo (90%+)'

        ELSE 'Con espacio disponible'

```

```

        END AS Estado_Carga

FROM RUTAS r

JOIN VEHICULOS v ON r.ID_Vehiculo = v.ID_Vehiculo

JOIN ASIGNACION_PEDIDOS_RUTA apr ON r.ID_Ruta = apr.ID_Ruta

JOIN DETALLE_PEDIDO dp ON apr.ID_Pedido = dp.ID_Pedido

JOIN PRODUCTOS prod ON dp.ID_Producto = prod.ID_Producto

GROUP BY r.ID_Ruta, r.Fecha_Programada, v.Placas, v.Capacidad_KG;

```

C. Movimientos de entrada al almacén del último mes, agrupados por proveedor.

```

SELECT

    prov.Nombre AS Proveedor,

    COUNT(DISTINCT e.ID_Entrada) AS Total_Recepciones,

    SUM(de.Cantidad_Recibida) AS Unidades_Totales_Entrantes

FROM ENTRADAS_ALMACEN e

JOIN PROVEEDORES prov ON e.ID_Proveedor = prov.ID_Proveedor

JOIN DETALLE_ENTRADA de ON e.ID_Entrada = de.ID_Entrada

WHERE e.Fecha >= CURRENT_DATE - INTERVAL '1 month'

GROUP BY prov.Nombre

ORDER BY Unidades_Totales_Entrantes DESC;

```

d.Rutas completadas de la semana con sus pedidos y si hubo incidencias.

```

SELECT

```

```

        r.ID_Ruta,

        r.Fecha_Programada,

        cond.Nombre AS Conductor,

        COUNT(apr.ID_Pedido) AS Total_Entregas,

        COUNT(apr.Incidencia) AS Total_Incidencias,

        STRING_AGG(apr.Incidencia, ' | ') AS Detalle_Incidencias

FROM RUTAS r

JOIN CONDUCTORES cond ON r.ID_Conductor = cond.ID_Conductor

JOIN ASIGNACION_PEDIDOS_RUTA apr ON r.ID_Ruta = apr.ID_Ruta

WHERE r.Estado_Ruta = 'Completada'

      AND r.Fecha_Programada >= DATE_TRUNC('week', CURRENT_DATE)

GROUP BY r.ID_Ruta, r.Fecha_Programada, cond.Nombre;

```

e. Valor total del inventario actual (stock × precio de compra) por categoría.

```

SELECT

        cat.Nombre AS Categoria,

        SUM(p.Stock_Actual) AS Unidades_En_Stock,

        TO_CHAR(SUM(p.Stock_Actual * p.Precio_Compra), 'FM$999,999,999.00')

AS Valor_Costo_Total

FROM PRODUCTOS p

JOIN CATEGORIAS cat ON p.ID_Categoria = cat.ID_Categoria

GROUP BY cat.Nombre

ORDER BY SUM(p.Stock_Actual * p.Precio_Compra) DESC;

```

f. El pedido más reciente de cada comercio.

```
SELECT DISTINCT ON (c.ID_Comercio)

    c.Nombre AS Comercio,

    z.Nombre AS Zona,

    p.ID_Pedido AS Ultimo_Pedido,

    p.Fecha_Solicitud,

    p.Estado

FROM COMERCIOS c

JOIN ZONAS z ON c.ID_Zona = z.ID_Zona

JOIN PEDIDOS p ON c.ID_Comercio = p.ID_Comercio

ORDER BY c.ID_Comercio, p.Fecha_Solicitud DESC;
```

4) Reportes Implementados como VIEWS (Esenciales ahora)

VIEW 1: Reporte de Zonas

```
CREATE OR REPLACE VIEW v_Reporte_Zonas AS

SELECT

    z.Nombre AS Zona,

    COUNT(p.ID_Pedido) AS Total_Pedidos,

    COUNT(*) FILTER (WHERE p.Estado = 'Entregado') AS
Entregas_Exitosas,

    COUNT(*) FILTER (WHERE p.Estado = 'Entregado con Incidencia') AS
Con_Incidencia,

    ROUND (

        (COUNT(*) FILTER (WHERE p.Estado IN ('Entregado', 'Entregado
con Incidencia'))::numeric
```

```

        / NULLIF(COUNT(p.ID_Pedido), 0)) * 100,

    2) || '%' AS Tasa_Efectividad

FROM ZONAS z

JOIN COMERCIOS c ON z.ID_Zona = c.ID_Zona

JOIN PEDIDOS p ON c.ID_Comercio = p.ID_Comercio

GROUP BY z.ID_Zona, z.Nombre

HAVING COUNT(p.ID_Pedido) > 5;

```

VIEW 2: Productividad de Conductores

```

CREATE OR REPLACE VIEW v_Productividad_Conductores AS

SELECT

    c.ID_Conductor,

    c.Nombre,

    COUNT(DISTINCT r.ID_Ruta) AS Rutas_Asignadas,

    COUNT(apr.ID_Pedido) AS Total_Pedidos_Entregados,

    ROUND(COUNT(apr.ID_Pedido)::numeric / NULLIF(COUNT(DISTINCT
r.ID_Ruta), 0), 1) AS Promedio_Pedidos_Por_Ruta,

    CASE

        WHEN (COUNT(apr.ID_Pedido)::numeric / NULLIF(COUNT(DISTINCT
r.ID_Ruta), 0)) > 15 THEN 'Elite (Alta Carga)'

        WHEN (COUNT(apr.ID_Pedido)::numeric / NULLIF(COUNT(DISTINCT
r.ID_Ruta), 0)) BETWEEN 5 AND 15 THEN 'Estándar'

        ELSE 'Bajo Rendimiento / Ruta Corta'

    END AS Clasificacion

FROM CONDUCTORES c

JOIN RUTAS r ON c.ID_Conductor = r.ID_Conductor

```



```

LEFT JOIN ASIGNACION_PEDIDOS_RUTA apr ON r.ID_Ruta = apr.ID_Ruta

WHERE r.Estado = 'Completada'

GROUP BY c.ID_Conductor, c.Nombre;

```

VIEW 3: Inventario

```

CREATE VIEW v_Inventario_Critico AS

WITH VentasMensuales AS (

    SELECT

        ID_Producto,

        SUM(Cantidad) as Total_Vendido_30Dias

    FROM DETALLE_PEDIDO dp

    JOIN PEDIDOS p ON dp.ID_Pedido = p.ID_Pedido

    WHERE p.Fecha_Solicitud >= CURRENT_DATE - INTERVAL '30 days'

    GROUP BY ID_Producto

)

SELECT

    prod.Nombre AS Producto,

    cat.Nombre AS Categoria,

    prod.Stock_Actual,

    COALESCE(vm.Total_Vendido_30Dias, 0) AS Ventas_Ultimo_Mes,

    CASE

        WHEN COALESCE(vm.Total_Vendido_30Dias, 0) = 0 THEN 'ALERTA:
Stock Muerto (Sin ventas)'

        WHEN (prod.Stock_Actual / (vm.Total_Vendido_30Dias / 30.0)) <
15 THEN 'CRITICO (< 15 Días)'

```

```

        WHEN (prod.Stock_Actual / (vm.Total_Vendido_30Dias / 30.0))
BETWEEN 15 AND 30 THEN 'Precaución'

        ELSE 'Saludable'

    END AS Estado_Inventario,

    CASE

        WHEN COALESCE(vm.Total_Vendido_30Dias, 0) > 0

        THEN ROUND((prod.Stock_Actual / (vm.Total_Vendido_30Dias /
30.0)), 1)

        ELSE NULL

    END AS Dias_Duracion_Estimada

FROM PRODUCTOS prod

JOIN CATEGORIAS cat ON prod.ID_Categoria = cat.ID_Categoria

LEFT JOIN VentasMensuales vm ON prod.ID_Producto = vm.ID_Producto

WHERE

    COALESCE(vm.Total_Vendido_30Dias, 0) = 0

    OR (COALESCE(vm.Total_Vendido_30Dias, 0) > 0 AND (prod.Stock_Actual
/ (vm.Total_Vendido_30Dias / 30.0)) < 15);

```

VIEW 4: La Financiera

```

CREATE OR REPLACE VIEW v_Rentabilidad_Proveedor AS

SELECT

    prov.Nombre AS Proveedor,

    COUNT(dp.ID_Pedido) AS Items_Vendidos,

    SUM(dp.Cantidad * prod.Precio_Compra) AS Costo_Total,

    SUM(dp.Subtotal) AS Venta_Total,

```

```

        (SUM(dp.Subtotal) - SUM(dp.Cantidad * prod.Precio_Compra)) AS
Ganancia_Bruta,

        ROUND (

            ((SUM(dp.Subtotal) - SUM(dp.Cantidad * prod.Precio_Compra)) /
NULLIF(SUM(dp.Subtotal),0)) * 100

            , 2) || '%' AS Margen_Porcentaje

FROM PROVEEDORES prov

JOIN PRODUCTOS prod ON prov.ID_Proveedor = prod.ID_Proveedor

JOIN DETALLE_PEDIDO dp ON prod.ID_Producto = dp.ID_Producto

GROUP BY prov.ID_Proveedor, prov.Nombre

ORDER BY Ganancia_Bruta DESC;

```

VIEW 5: Enfocada a Gestión de Activos

```

CREATE OR REPLACE VIEW v_Utilizacion_Flota AS

SELECT

    v.Placas,

    v.Capacidad_KG,

    v.Estado AS Estado_Actual,

    COUNT(r.ID_Ruta) AS Rutas_Mes_Actual,

    CASE

        WHEN COUNT(r.ID_Ruta) > 25 THEN 'Sobrecargado (Uso Diario)'

        WHEN COUNT(r.ID_Ruta) BETWEEN 10 AND 25 THEN 'Uso Óptimo'

        ELSE 'Subutilizado / Parado'

    END AS Analisis_Uso

FROM VEHICULOS v

```

```
LEFT JOIN RUTAS r ON v.ID_Vehiculo = r.ID_Vehiculo

    AND r.Fecha_Programada >= DATE_TRUNC('month', CURRENT_DATE)

GROUP BY v.ID_Vehiculo, v.Placas, v.Capacidad_KG, v.Estado;
```